dependency management system solves the issue that you've read about in *The Problem of Chained Inclusions*, on page 181.

After rolling all your dependent concerns into your includer's chain of ancestors, you still have a couple of things to do. First, you must add *yourself* to that chain of ancestors, by calling the standard Module.append_features with super. Finally, don't forget what this entire machinery is for: you have to extend the includer with your own ClassMethods module, like the include-and-extend trick does. You need Kernel#const_get to get a reference to ClassMethods, because you must read the constant from the scope of self, not the scope of the Concern module, where this code is physically located.

Concern also contains some more functionality, but you've seen enough to grasp the idea behind this module.

Concern Wrap-Up

ActiveSupport::Concern is a minimalistic dependency management system, wrapped into a single module with just a few lines of code. That code is complicated, but using Concern is easy, as you can see by looking into Active Model's source:

gems/activemodel-4.1.0/lib/active_model/validations.rb

```
module ActiveModel
  module Validations
    extend ActiveSupport::Concern
    # ...

    module ClassMethods
      def validate(*args, &block)
        # ...
```

Just by doing the above, ActiveModel::Validation adds a validate class method to ActiveRecord::Base, without worrying about the fact that ActiveRecord::Validation happens to be in the middle. Concern will work behind to scenes to sort out the dependencies between concerns.

Is ActiveSupport::Concern too clever for its own good? That's up to you to decide. Some programmers think that Concern hides too much magic behind a seemingly innocuous call to include, and this hidden complexity carries hidden costs.[1] Other programmers praise Concern for helping to keep Rails' modules as slim and simple as they can be.

Whatever your take on ActiveSupport::Concern, you can learn a lot by exploring its insides. Here is one lesson I personally took away from this exploration.

1. http://blog.coreyhaines.com/2012/12/why-i-dont-use-activesupportconcern.html